

Slide 14A & 14B — Backend & Deployment Architecture (Fact-Checked)

All technical claims below have been verified against the actual codebase: `api.py`, `core_ai.py`, `Dockerfile`, and `requirements.txt`.

Slide 14A: Backend Architecture & Database Integration

Slide Title:

Backend Architecture & Database Integration

Visual Layout (Left 50% Diagram / Right 50% Bullets):

Left Column — Diagram: Draw a vertical flow diagram with these blocks:



Caption: Figure 14A — FastAPI request pipeline with validation, logging, and resilient DB pooling.

Right Column — Slide Bullet Points:

- **Framework:** Python **FastAPI** — asynchronous ASGI web framework.
- **Server:** **Uvicorn** — lightweight async server that handles concurrent requests without thread blocking.
- **Input Validation:** **Pydantic** schemas enforce strict type and length constraints at the API boundary (e.g., exactly 42 survey answers, non-empty text).
- **Request Logging:** Custom HTTP middleware logs every request's method, URL path, status code, and latency (in ms) for debugging and monitoring.
- **Database:** **SQLAlchemy ORM** connecting to **Supabase PostgreSQL** with:
 - `connect_timeout = 5 seconds`
 - `pool_pre_ping = True` (tests connections before use)
- **Graceful Startup:** Database table creation is wrapped in an **8-second async timeout** — if the DB is unreachable, the server starts anyway without crashing.

Speaking Script — Slide 14A

"Let's examine the backend architecture of SafeSpace — the core engine behind all our AI computations and data persistence.

We built our backend using **Python FastAPI**, which is an asynchronous ASGI framework. Unlike traditional synchronous frameworks like Flask, FastAPI leverages Python's `async/await` event loop, allowing the server to process multiple client requests concurrently without blocking. We run it on the **Uvicorn** ASGI server.

Every incoming request passes through two middleware layers. First, a **CORS middleware** configured to allow all origins, which is necessary because our Flutter mobile client and web client send requests from different domains. Second, a **custom request-logging middleware** that records the HTTP method, URL path, response status code, and execution latency in milliseconds for every single request. This gives us full observability into backend performance.

Before any request reaches our AI models or database, it passes through **Pydantic validation schemas**. For example, our `AnalyzeRequest` schema enforces that the survey answer array must contain exactly 42 integers, and the text input must be at least one character long. If the client sends malformed data, FastAPI rejects it immediately with a 422 error — the request never touches the models or the database.

For data persistence, we use **SQLAlchemy ORM** to connect to a cloud-hosted **Supabase PostgreSQL** database. Because cloud connections can drop, we configured the connection pool with two resilience settings: a **5-second connection timeout**, and **pool_pre_ping set to True**. This pre-ping setting forces SQLAlchemy to issue a lightweight test query — essentially a `SELECT 1` — before handing a connection to our application code. If the test fails, it silently discards the stale connection and opens a fresh one. This prevents database-related crashes.

Additionally, during server startup, we create all database tables inside an **8-second async timeout**. If the database happens to be temporarily unreachable at boot time, the server starts anyway and logs a warning rather than crashing. This ensures the container remains healthy even during brief database outages.

Now let's look at how we deploy this to production."

🗒 Jury Q&A — Slide 14A

Q: Why did you choose FastAPI over Django or Flask?

"Flask is synchronous by default — each request blocks a thread. Django is a full-stack framework with built-in ORM, admin panel, and templating — all unnecessary overhead for a pure API backend. FastAPI is purpose-built for APIs: it gives us async performance, automatic OpenAPI/Swagger documentation, and Pydantic validation out of the box with minimal boilerplate."

Q: What does `pool_pre_ping=True` actually do under the hood?

"It's SQLAlchemy's pessimistic disconnect handler. Before the pool returns a connection to our code, it runs a `SELECT 1` test. If the test fails because the connection has been dropped by the database server or a network timeout, the pool discards that connection and creates a new one transparently. Our application code never sees the error."

Q: How do you secure the database and API?

"Three layers. First, all traffic between the Flutter client and our API uses HTTPS. Second, user passwords are hashed using SHA-256 before storage — we never store plaintext passwords. Third, all database queries filter by `user_id`, so one user cannot access another user's journal entries or analysis history."

Q: How do you handle a malformed request — for example, if someone sends 41 answers instead of 42?

"Pydantic validates the request before our endpoint code runs. The `AnalyzeRequest` schema specifies `min_items=42`, `max_items=42` for the survey array. If the client sends 41 answers, FastAPI automatically returns a 422 Unprocessable Entity error with a clear message describing the validation failure. Our model code never sees invalid input."

Q: What happens if the database is completely down?

"The server still starts and serves AI predictions. During startup, the database initialization is wrapped in an 8-second timeout. If it times out, we log a warning and proceed. Additionally, every database write in the analyze endpoint is wrapped in a try/except — if the save fails, the API still returns the analysis results to the user. We prioritize user experience over data logging."

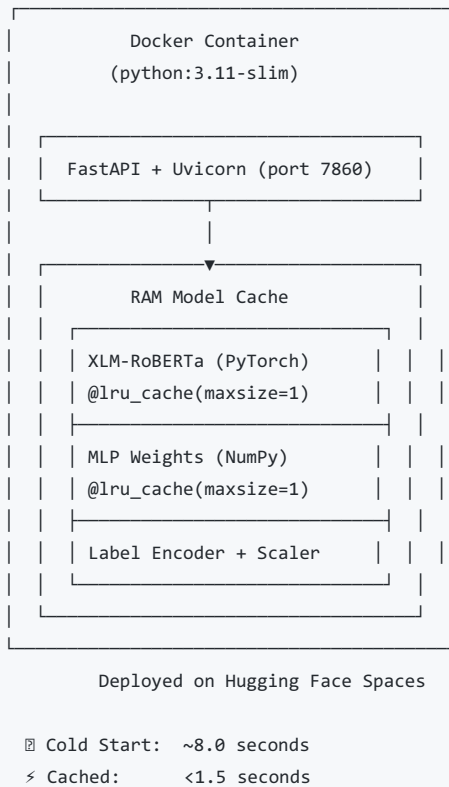
🗒 Slide 14B: Cloud Deployment & Model Optimization

Slide Title:

Cloud Deployment & Model Optimization

Visual Layout (Left 50% Diagram / Right 50% Bullets):

Left Column — Diagram: Draw a deployment flow with a latency comparison:



Caption: Figure 14B — Dockerized serverless deployment with in-memory transformer caching.

Right Column — Slide Bullet Points:

- **Containerization:** Full environment packaged in a **Docker** image based on `python:3.11-slim` with `build-essential` for C extension compilation.
- **Hosting:** Deployed on **Hugging Face Spaces** — free-tier serverless container hosting with automatic restarts.
- **Model Source:** XLM-RoBERTa loaded from **Hugging Face Hub** (`AliSakr9997/Mental-XLMR-Model`) or from a local `mental_xlmr_final/` directory if weights are pre-bundled.
- **Caching:** Both the NLP model and the MLP survey model are wrapped in `@lru_cache(maxsize=1)` — loaded once on first request, then served from RAM for all subsequent calls.
- **Inference Optimization:** Gradient tracking disabled via `torch.no_grad()`, text truncated at 192 tokens, reducing memory and CPU usage.
- **Latency:** Cold start ~8.0s → Cached inference <1.5 seconds.

🗣️ Speaking Script — Slide 14B

"Now let's look at how we deploy this backend to production and how we optimized model performance.

We packaged the entire backend into a **Docker container** based on the `python:3.11-slim` image. This container bundles our exact versions of **PyTorch**, **Transformers**, **NumPy**, and all other dependencies. This eliminates the 'it works on my machine' problem — the exact same environment runs locally and in production.

We deploy this container to **Hugging Face Spaces**, which provides free-tier serverless container hosting. When we push code to the repository, Hugging Face automatically rebuilds and redeploys the container. The server starts on port 7860 running Uvicorn.

Our primary technical challenge was **model loading latency**. The XLM-RoBERTa transformer model is loaded from Hugging Face Hub — the repository `AliSakr9997/Mental-XLMR-Model`. On a cold start, downloading and initializing this model takes approximately **8 seconds**, which is unacceptable for a real-time application.

We solved this with `@lru_cache` decorators. Both the `load_xlmr()` function — which loads the tokenizer, transformer model, and label encoder — and the `load_survey()` function — which loads the MLP weights and scaler — are decorated with `@lru_cache(maxsize=1)`. This means the first request triggers a full model load, and every subsequent request reuses the same objects already in RAM.

We also disabled PyTorch's gradient computation by running inference inside a `torch.no_grad()` context block. Since we're only doing forward-pass prediction and not training, disabling gradients eliminates unnecessary memory allocation for the computational graph. Combined with truncating input text to a maximum of **192 tokens**, this keeps the CPU and memory footprint manageable on Hugging Face's free tier.

The result: our prediction latency dropped from **8.0 seconds on cold start to under 1.5 seconds** for all active requests. This gives our users a near-instant assessment experience.

I will now hand over to my colleague to discuss our Unique Value Proposition — the Crisis Override and Safety Net system. Thank you."

🗣️ Jury Q&A — Slide 14B

Q: Why Docker instead of deploying directly to a cloud VM or PaaS like Heroku?

"Machine learning backends have complex binary dependencies — PyTorch, TensorFlow, SentencePiece, psycpg2. These often conflict across OS versions and

Python environments. Docker guarantees that the exact binary environment we test locally is the exact one running in production. Hugging Face Spaces natively supports Docker, so deployment is as simple as a git push."

Q: How does `@lru_cache(maxsize=1)` work for model caching?

"It's Python's built-in function memoization decorator. When `load_xlmr()` is called the first time, it downloads the model, initializes the tokenizer, and loads the label encoder. The decorator caches this return value. On every subsequent call, it returns the cached objects directly from memory without re-executing the function. `maxsize=1` means we only store one cached result, which is all we need since the model never changes at runtime."

Q: What happens if the Hugging Face container goes to sleep due to inactivity?

"Hugging Face Spaces free tier does put containers to sleep after a period of inactivity. When the next request arrives, the container wakes up and the models reload — that's the 8-second cold start. For a production deployment with guaranteed uptime, we would upgrade to a paid persistent Space or migrate to a cloud provider with always-on instances."

Q: Why is the max token length set to 192 instead of the standard 512?

"Journal entries and text inputs in our application are typically short — a few sentences. Setting `max_length` to 192 reduces tokenization and attention computation time without losing meaningful content. In our testing, journal entries rarely exceeded 100 tokens. The 192 limit provides headroom while keeping inference fast on CPU."

Q: How do you handle model versioning? What if you retrain the model?

"The model source is controlled by an environment variable `HF_MODEL_ID`. If we retrain and publish a new model, we update this variable and restart the container. The `analyses` table also has `model_version` and `app_version` columns, which would allow us to track which model version produced each assessment — useful for A/B testing or auditing."

Q: You have both TensorFlow and PyTorch in `requirements.txt`. Why?

"The MLP survey model was originally trained using TensorFlow/Keras, and the weight export pipeline uses TensorFlow utilities. The NLP model uses PyTorch. In production, the MLP inference runs purely on NumPy — we don't actually invoke TensorFlow at runtime. In a future optimization, we could remove TensorFlow from the production requirements to reduce the Docker image size."